Graph-based data modeling system with LLM intent ro... Shared by Developer

This is a copy of a chat between Claude and Developer. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here. Report

System: You are a principle enginner with 10+ years of Experience in AI based systems and LLM systems I need you to guide me step By step through the development process of this Graph-Based Data modeling and Query system...

What I want?

1. I want to make my system robust, in terms of architecture and functioning
2. I want to use design patterns to build this project and make my project look like someone expert has build it
3. I want end to end working architecture walkthrough of the project so that antigravity.google can write the code on my behalf and define all the edge cases and corner cases, but behind the scenes it should hold my architectural decisions..

What i understood? (validate my understanding)
1. the flow of the project is like ->

```
Customer → Sales Order → Delivery → Billing → Journal Entry
```

1. Everything rotates around the , customer just we gonna decode the user interpretation and map it to the elements of the collection and build the query
2. LLM's should just bother about decoding the user intent and the type of data it wants rest everything will be handled by query generated by us, we will have store of queries which we will use to return the response and these responses will be again feed to the LLM to get constructive LL based answers..
3.

```
User Input
    ↓ LLM (reads meaning, returns intent JSON)
DSL Object (validated Pydantic model)
    ↓ Intent Router (pure Python)
Execution Engine (deterministic MongoDB pipelines)
    ↓ Raw Data
LLM (reads data, writes human prose)

    ↓
```

```
Final Answer
```

this is the flow that I am thinking of
5. We need to anticipate every failure modes and restrict our AI to respond under the given Domain. As this project suggest us to stay in domain specified and all domain specific things are built around some restrictions..

So how is my project level understanding? I need you to have an interactive session with me where we will discuss LLD for now as, we would first make things work and later on discuss about how can we optimize the performance and expand our knowledge through wide spectrum. Dont code, feel free to ask questions?

Show less

Your understanding is quite solid for where you are in the process. Let me give you quick feedback on each point, and then I have some important clarifying questions before we dive into LLD.

Your understanding — validated:

Points 1-3 are directionally correct. The linear flow (Customer → Sales Order → ... → Journal Entry) is one path, but the "graph-based" part of your project title suggests there are cross-links and non-linear traversals too, which we need to discuss. Point 4 (your pipeline diagram) is the strongest part — that's a clean separation of concerns. Point 5 about domain restriction is critical and shows maturity in thinking.

One thing I want to flag early: You said "store of queries" — this is a design decision with massive downstream consequences. Whether you go with pre-built template pipelines vs. dynamically assembled pipelines vs. a hybrid changes your entire routing and validation layer. We'll need to nail this down.

Now, before I can guide you on LLD with confidence, I need to clear up a few things: Also, while you're answering those — a few more things I need to understand in your own words:

1. **The "graph" in your title** — your flow shows a linear chain, but real business data has branches (e.g., one Sales Order → multiple Deliveries, one Customer → multiple Sales Orders with different statuses). How complex are these relationships in your data?
2. **Scale context** — are we talking hundreds of customers or hundreds of thousands? This changes whether we index-first or optimize-later.
3. **The LLM choice** — are you locked into a specific provider (OpenAI, Anthropic

3. **The LLM choice** — are you locked into a specific provider (OpenAI, Anthropic, open-source), or is this provider-agnostic by design?
4. **What does "antigravity.google" refer to?** — is this a coding agent you want to hand off the implementation to after we finalize architecture?

Take your time. The better we align here, the cleaner our LLD session will be.

[User denied question request]

Files hidden in shared chats

1. relationships are quite complex, like direct relationships using foreignkey, multi-hop relationships and so on. i have provided you with my ER diagram of my DB you can see on the relationship pattern.
2. For now lets consider it for 100's of customer only. Because we have a limited timeframe

Show more

Good — this changes the picture significantly. Let me process everything you've shared before we go further.

What I now understand clearly:

Your ER diagram is textbook SAP O2C data — about 18 entities with header/item patterns (sales_order_headers ↔ sales_order_items, billing_document_headers ↔ billing_document_items, etc.). The relationships are through document number references, not traditional FKs with constraints — which is very SAP-like. The BAES document is strong architectural thinking.

But I have critical conflicts to resolve before LLD makes sense:

Your BAES document says MongoDB aggregation pipelines and mentions

Your BAES document says MongoDB aggregation pipelines and mentions GraphStore with Neo4j as an option. But you just told me PostgreSQL. These are fundamentally different execution engines — the entire query layer, indexing strategy, and pipeline design changes based on this choice.
Also, two more things I need clarity on while you answer those:

On your LLM auto-rotation idea — this is a smart cost optimization, but it introduces a design question: different free-tier LLMs (Gemini, Groq, Mistral via OpenRouter) have wildly different structured output reliability. Your intent extraction layer needs to produce valid Pydantic objects every time. Are you okay with a fallback strategy where if a weaker model fails structured extraction, it retries with a more reliable one? Or do you want strict round-robin regardless of capability?

On your ER diagram — I notice the join paths are implicit (e.g., `billing_document_items.reference_sd_document` → `sales_order_items.sales_order`). These are SAP-style "soft references" not enforced FK constraints. When we model this in PostgreSQL, do you want to enforce referential integrity with actual FK constraints, or keep them soft and handle integrity at the application layer? This matters for your graph traversal reliability.

Take your time answering — these decisions are load-bearing for the entire LLD.

Q: Final call on storage — which setup are you committing to?
A: PostgreSQL as primary + Neo4j for graph traversals

Q: Your BAES doc mentions deterministic pipelines. For PostgreSQL, how do you want query construction to work?
A: whats your choice, see at this stage making querying process dynamic is quite complicated because we need to keep our LLM aware of the fields, tables and relationshipd and stil there are chances of making mistake in generating query. I need your guidance i am not completely sure

Q: Your message ended with 'My LLD designing, what I am thinking:' — did you have more to share?
A: I have shared an artifact , where i have presented my design mindset

Good. I've now absorbed your BAES document, the ER diagram, and your preferences. Let me give you my strong recommendation on the query construction question, then flag one concern about your dual-DB choice before we start LLD.

My recommendation on query construction: Template-first with parameterized

My recommendation on query construction: Template first with parameterized slots.

Here's my reasoning. Look at your intent types — `TopProductsByBillingIntent`, `FlowTraceIntent`, `BrokenFlowIntent`, `EntityMetadataIntent`. These are finite, known operations. You're not building a general-purpose "ask anything about SQL" system. You're building a domain-locked system where the LLM's only job is to figure out *which* operation the user wants and *what parameters* they're providing.

So the architecture becomes: the LLM never sees SQL or Cypher. Ever. It only produces a structured DSL object like `FlowTraceIntent(entity="sales_order", document_id="0001234567", depth=3)`. Your handler picks up a pre-built, tested, battle-hardened query template and injects those parameters. This gives you deterministic correctness, full testability without LLM involvement, and zero risk of SQL injection or hallucinated joins.

For the rare "I didn't anticipate this query shape" scenario, you add a fallback that says "I can't answer that yet" rather than attempting dynamic generation. You expand templates over time based on what users actually ask.

Now — the concern about PostgreSQL + Neo4j:

This is a legitimate architectural choice, but it doubles your operational surface. Before I can design the LLD, I need to understand the split clearly.
One more important thing while you answer — about your ER diagram's **soft references**: fields like `billing_document_items.reference_sd_document` pointing to `sales_order_headers.sales_order`. In SAP, these are just string fields, not enforced FKs. When we set up PostgreSQL, I'd recommend enforcing these as actual foreign keys where the relationship is guaranteed (header → items), and keeping them as soft lookups where SAP allows orphaned references (cross-document chains like billing → sales order). This gives you data integrity where it matters without fighting SAP's inherent looseness.

Once you answer these three, I'll have enough to start drawing out the LLD layer by layer. We're close.

> . In SAP, these are just string fields, not enforced FKs. When we set up PostgreSQL, I'd recommend enforcing these as actual foreign keys where the relationship is guaranteed (header → items), and keeping them as soft lookups where SAP allows orphaned references (cross-document chains like billing → sales order). This gives you data integrity where it matters without fighting SAP's inherent looseness.
>
> yes your thinking is right we can keep soft lookups

okay i adher to you recommendation but i need Dynamic PostgreSQL

Okay rather to your recommendation but need Dynamic PostgreSQL query builder as python code only based on rules interpreted by LLM if those rules are out of context than we can give errors this is more like a hybrid structure our LLM defines the rules and our python generated the query based on the rules generated by LLM by interpreting the user query and so on...

Now I'm tracking exactly where your head is. And honestly, this is the right instinct — let me validate and refine it.

What you're describing is a Constrained Rule-Based Query Builder, and it's a strong middle ground. Here's why I agree:

Pure templates are too rigid — you'd be forever adding new templates for every slight variation ("top 5 products" vs "top 10 products by region" vs "top products this quarter"). Pure dynamic SQL generation by LLM is too dangerous — hallucinated joins, wrong column names, injection risks.

Your hybrid means the LLM's output is never SQL. It's a structured rule set — essentially a validated Pydantic object that says things like "I want to aggregate, on this entity, filtering by these fields, sorting by this, with this limit." Then a deterministic Python query builder reads those rules and assembles safe, tested SQL using only known tables, known columns, and known join paths. If any rule references something outside your domain schema, the builder rejects it before a single query hits the database.

This is a pattern I've seen work well in production systems. We'll design it properly in LLD.

But I still need answers to my three pending questions before I can start — they're blocking decisions:
These three answers will lock in the last architectural decisions I need. Once you answer, I'll lay out the full LLD — layer by layer, pattern by pattern — covering your domain model, the DSL schema design, the rule-based query builder internals, the dual-store sync strategy, the LLM adapter with rotation, and the supervision/guardrail chain. No code, just clean architectural decisions with rationale.

Q: How do PostgreSQL and Neo4j coexist? Who owns the data?
A: PostgreSQL is source of truth, Neo4j is synced projection

Q: Is Neo4j driven by graph visualization needs or multi-hop query expressiveness?
A: multi-hop query expressiveness

Q: LLM auto-rotation strategy?

Q: LLM auto-rotation strategy?
A: Fallback-based: try best model, retry on failure

Excellent. All decisions are locked. Let me lay out where we stand before we start LLD.

Locked-in architectural decisions:

Decision	Choice
Primary store	PostgreSQL (source of truth)
Graph store	Neo4j (synced projection, multi-hop queries)
Query strategy	Constrained Rule-Based Builder (LLM → rules → Python → SQL/Cypher)
LLM strategy	Fallback-based rotation with health tracking
FK strategy	Hard FKs for header↔item, soft lookups for cross-document
Backend	FastAPI (async)
Scale	Hundreds of customers (v1)
Framework	BAES (Blueprint, Assembly, Execution, Supervision)

Now let me present the full LLD architecture, and then we'll drill into each layer one by one.
Excellent — all decisions are now locked. Let me present the full system architecture, then we'll drill into each layer.

Here's what we've committed to:

Your BAES framework maps cleanly onto this. The LLM produces structured *rules* (not queries), your Python layer builds safe SQL/Cypher from those rules against a known schema registry, and supervision watches everything. PostgreSQL owns the truth, Neo4j gets a synced projection for multi-hop traversals that would be painful as recursive SQL joins.

Log in to see interactive visuals

Each box in the diagram is clickable if you want to jump straight into a specific layer. But let me give you the 30-second overview of how each layer connects, then you pick where we start drilling.

The request lifecycle in one paragraph: A user query hits FastAPI, which passes it to

The request lifecycle in one paragraph: A user query hits Fast API, which passes it to the LLM intent parser. The parser tries the best available model (say Gemini), and if that fails, falls back to the next provider. It outputs a structured Pydantic DSL object — not SQL, not Cypher, just typed rules. That DSL runs through the guardrail chain (is this in-scope? are the fields valid? is the entity allowed?). If approved, the intent router dispatches to the right handler via Strategy pattern. The handler feeds rules to the query builder, which assembles safe SQL (for aggregation, filtering, metadata) or Cypher (for multi-hop traversals like "trace sales order 1234 through delivery to billing to payment"). Results come back as raw data, get passed to a second LLM call that produces a human-readable narrative, and the structured response ships back to the client. Supervision observes every step.

The six LLD layers we need to design:

1. **DSL schema design** — what exactly does the LLM output? The "rules" contract.
2. **Query builder internals** — how rules become safe SQL/Cypher against a schema registry.
3. **PostgreSQL schema + Neo4j graph model** — how your 18 SAP entities map to both stores, plus the sync mechanism.
4. **LLM adapter + fallback chain** — provider abstraction, health tracking, retry logic.
5. **Intent router + handlers** — Strategy/CoR pattern for dispatch.
6. **Supervision** — observer events, guardrail chain, audit trail.

My recommendation is to start with Layer 1 (DSL schema) because it's the contract everything else depends on. If we get the DSL wrong, every downstream layer breaks. But it's your call.
Once you rank them, we'll go deep into each — discussing the internal structure, design patterns, edge cases, failure modes, and how it connects to the layers above and below it. No code, just clean architectural reasoning.

Q: Rank the layers in the order you want to discuss them (drag to reorder):
A: Ranked: 1. DSL schema design (the rules contract), 2. Query builder internals, 3. PostgreSQL + Neo4j data model and sync, 4. LLM adapter + fallback chain, 5. Intent router + handlers, 6. Supervision + guardrails

Perfect ordering. Let's go deep on Layer 1 — the DSL schema design. This is the most important decision in the entire system because it's the contract between two

unreliable boundaries: the LLM (which can hallucinate) and the database (which will

unreliable boundaries: the LLM (which can hallucinate) and the database (which will crash on bad input). The DSL is your firewall between them.

The core principle: The LLM never produces queries. It produces *structured intent objects* — think of them as validated instruction packets. Each packet says "what I want" in typed fields, and your Python layer decides "how to get it."

Let me walk you through the three parts of this layer: the Schema Registry (what's allowed), the Intent Taxonomy (what the LLM can request), and the Filter Model (how conditions work).

Part 1: The Schema Registry

Before we even talk about intents, we need a single source of truth that answers: what entities exist, what fields do they have, what types are those fields, and what are the legal join paths between entities? This registry is referenced by three things — the LLM prompt (so it knows what's available), the DSL validation (so it can reject invalid field references), and the query builder (so it knows how to join tables).

Log in to see interactive visuals

Looking at your ER diagram, the registry would catalog all 18 entities with their fields and types. For example, `sales_order_headers` has `sales_order` (string, PK), `total_net_amount` (decimal, aggregatable), `creation_date` (date, filterable), etc. The join paths section is critical — it encodes things like "sales_order_items connects to billing_document_items via `reference_sd_document` = `sales_order`". This is where your graph structure lives in declarative form.

Why this matters: If the LLM says "filter by `invoice_status`" and that field doesn't exist in the registry, validation kills it immediately. If it says "join orders to payments directly," the registry knows that's a 3-hop path (order → delivery → billing → payment), not a direct join. The registry is your safety net.

Part 2: The Intent Taxonomy

Based on your O2C domain, I see six core intent types that cover the question space. Let me walk through each one with example user questions that would trigger them.

Log in to see interactive visuals

Let me walk through each intent type with real user questions from your O2C domain so this becomes concrete.

EntityLookupIntent — single document retrieval. User says: "Show me sales order 0000012345" or "What's the status of delivery 80001234?" The LLM produces

```
intent_type: "entity_lookup"  entity_type: "sales_order_headers"
```

`intent_type: "entity_lookup"`, `entity_type: "sales_order_headers"`, `identifier: "0000012345"`. Simple, deterministic, one-row result.

EntityListIntent — filtered listing. User says: "Show me all orders for customer 1000045 this quarter" or "Which customers are blocked?" The LLM produces `intent_type: "entity_list"`, `entity_type: "business_partners"`, `filters: [{field: "business_partner_is_blocked", operator: "eq", value: true}]`, `limit: 50`. Notice the filter references a real field from your ER diagram — the registry validates this exists.

AggregationIntent — metrics and rankings. User says: "Top 10 products by billing amount" or "Revenue by sales organization last quarter." The LLM produces `intent_type: "aggregation"`, `entity_type: "billing_document_items"`, `measure: "net_amount"`, `agg_fn: "sum"`, `group_by: ["material"]`, `sort_by: "sum_net_amount"`, `sort_order: "desc"`, `limit: 10`.

FlowTraceIntent — this is where Neo4j earns its keep. User says: "Trace sales order 12345 from order through delivery to payment." The LLM produces `intent_type: "flow_trace"`, `start_entity: "sales_order_headers"`, `start_id: "0000012345"`, `target_entity: "payment_accounts_receivable"`, `max_depth: 4`. This becomes a Cypher traversal query in Neo4j because following the chain (order → order items → delivery items → billing items → journal entries → payments) would be a nightmare in SQL with 5+ recursive joins.

BrokenFlowIntent — the operations gold mine. User says: "Which orders don't have deliveries yet?" or "Find billing documents without payments." The LLM produces `intent_type: "broken_flow"`, `source_entity: "sales_order_headers"`, `expected_target: "outbound_delivery_headers"`, `filters: [{field: "creation_date", operator: "gte", value: "2025-01-01"}]`. This is a LEFT JOIN ... WHERE target IS NULL pattern in SQL, or a graph pattern match with missing edges in Cypher.

OutOfScopeIntent — the safety valve. User says: "What's the weather today?" or "Write me a poem about invoices." The LLM produces `intent_type: "out_of_scope"`, `reason: "General knowledge question unrelated to O2C data"`, `suggestion: "Try asking about orders, deliveries, billing, or payments."` This is not a failure — it's a designed behavior.

Part 3: Critical design decisions I need your input on

Before we move to Layer 2, there are three DSL design choices that affect everything downstream:

These answers will finalize the DSL contract, and then we move straight into Layer 2 — the query builder internals, which is where your "rules → safe SQL/Cypher" engine lives. Take your time with these.

Q: Should the LLM produce real database field names or semantic aliases? Aliases are safer (LLM doesn't hallucinate column names) but add a mapping layer.
A: LLM uses semantic aliases (order_number, amount) mapped to real fields

Q: Should one user query always map to exactly one intent? Or can a complex question decompose into multiple intents?
A: Allow compound: e.g. 'Show top products AND trace the best one'

Q: When the LLM produces a field/entity that doesn't exist in the registry, what should happen?
A: auto fic obvious typos if confident else ask clarification question

Strong choices — and each one has downstream consequences that I want to make explicit before we lock the DSL and move on.

Decision 1: Semantic Aliases — the implications

This is the right call. It means the LLM never sees `billing_document_items.reference_sd_document` — it sees `billing_item.order_reference`. This gives you three advantages: the LLM's prompt becomes readable, you can rename DB columns without touching any prompts, and fuzzy matching becomes easier on natural language terms than on SAP technical names.

But it means we need an **Alias Registry** as a sub-component of the Schema Registry. Every alias maps to exactly one `(table, column)` pair, and every `(table, column)` pair can have multiple aliases. For example, `order_number` and `sales_order_number` both resolve to `sales_order_headers.sales_order`. The query builder never sees aliases — by the time it runs, everything is resolved to real column references.

Decision 2: Compound Intents — this is the ambitious one

When a user says "Show me top 5 products by billing amount and trace the flow of the highest one," that's two intents with a *dependency*: the second intent needs a result from the first. This means the LLM output isn't just a list of intents — it's a mini execution plan with data flow between steps.

Log in to see interactive visuals

The compound intent structure needs careful constraints. The LLM outputs a

The compound intent structure needs careful constraints. The LLM outputs a `CompoundIntent` containing an ordered list of steps, where each step is a regular intent (aggregation, lookup, trace, etc.) plus an optional `depends_on` field referencing a previous step's ID. The execution engine runs them sequentially, substituting result references as it goes. We cap it at 3 steps to prevent runaway chains, and if any step fails, we return partial results with an explanation rather than failing the entire request.

The important nuance here: the LLM doesn't decide *how* to chain results. It just says "step 2 uses the first result from step 1." Your Python execution engine resolves that reference after step 1 completes. The LLM stays declarative, the engine stays deterministic.

Decision 3: Soft Correction with Clarification Fallback

This gives us a two-tier resolution strategy for invalid references. Here's how I'd design it:

The DSL validator runs every field and entity reference through the alias registry. If there's no exact match, it falls to **fuzzy matching** (Levenshtein distance against all known aliases). If the best match is above a confidence threshold (say 85% similarity), auto-correct silently and log it. If the best match is between 50-85%, return a clarification question: "Did you mean `billing_document` or `billing_cancellation`?" Below 50%, reject as out-of-scope.

The key detail: auto-corrections must be logged in the audit trail so you can see if the LLM is consistently producing near-miss aliases. That's a signal to either add those as official aliases or fix the prompt.

Final DSL Contract Summary

Let me consolidate everything we've decided for Layer 1 before we move on:

Log in to see interactive visuals

That's Layer 1 locked. Before we move to Layer 2 (Query Builder Internals), I want to make sure we're aligned.
One thought while you decide: the alias registry is something your coding agent will need heavy guidance on. The mapping between semantic names and SAP field names is domain knowledge that only you have — for example, knowing that "order amount" should map to `sales_order_headers.total_net_amount` and not `sales_order_items.net_amount` (which is a line-item amount, not the order total). When we get to writing the instructions for your agent, this alias dictionary will be one of the most important artifacts.

Q: How are you feeling about the DSL layer? Ready to move on or need to dig deeper anywhere?
A: All clear — move to Layer 2: Query Builder

Let's get into the engine room. Layer 2 is where your DSL rules become safe, executable queries. This is the layer that makes your "constrained rule-based builder" real.

The query builder has one job: take a fully validated, alias-resolved intent object and produce a parameterized query — SQL for PostgreSQL, Cypher for Neo4j — without ever accepting raw user input into the query string. Everything goes through parameters.

Three sub-systems to design here: the Store Router (which database handles this intent?), the SQL Builder (for PostgreSQL), and the Cypher Builder (for Neo4j).

Part 1: The Store Router

This is the first decision point inside the query builder. Not every intent goes to the same database.

Log in to see interactive visuals

The logic is straightforward: if the intent operates on a single entity or needs aggregation (SUM, COUNT, GROUP BY), PostgreSQL handles it — that's what relational databases are built for. If the intent requires following a chain of relationships across multiple document types (order → delivery → billing → payment), Neo4j handles it because recursive multi-hop traversals are its strength. For compound intents, each step is routed independently — step 1 might go to PostgreSQL, step 2 to Neo4j.

There's a subtle edge case here: `BrokenFlowIntent` could technically be done in PostgreSQL with a LEFT JOIN ... WHERE NULL pattern for single-hop gaps (orders without deliveries). But for multi-hop gaps (orders that reached delivery but never got billed), Neo4j is cleaner. The router checks the **hop distance** between `source_entity` and `expected_target` in the schema registry's join path graph. One hop → SQL. Two or more hops → Cypher.

Part 2: The SQL Builder

This is the workhorse for 70-80% of your queries. Let me walk through how a resolved

This is the workhorse for 70-80% of your queries. Let me walk through how a resolved intent becomes a parameterized SQL statement.

Log in to see interactive visuals

Let me walk through a concrete example so this becomes real. User asks: **"Show me top 10 products by billing amount for customer ACME this quarter."**

The LLM produces an AggregationIntent with aliases: `entity: "billing_items"`, `measure: "amount"`, `agg_fn: "sum"`, `group_by: ["product"]`, `filters: [{field: "customer_name", operator: "like", value: "ACME%"}, {field: "billing_date", operator: "gte", value: "2026-01-01"}]`, `sort_by: "sum_amount"`, `sort_order: "desc"`, `limit: 10`.

After alias resolution, the builder sees: base table is `billing_document_items`, measure is `net_amount`, group by `material`. But here's the interesting part — `customer_name` doesn't live on `billing_document_items`. It lives on `business_partners`. And `billing_date` resolves to `billing_document_headers.billing_document_date`.

This is where the **Join Resolver** kicks in.

Part 3: The Join Resolver — the brain of the SQL builder

The join resolver looks at all the tables referenced across SELECT, WHERE, and GROUP BY, determines the shortest join path between them using the schema registry's join path graph, and assembles the JOIN chain automatically.

Log in to see interactive visuals

The join resolver uses BFS (breadth-first search) on the schema registry's join path graph. Every pair of tables that can be joined has an entry like: `billing_document_items → billing_document_headers via billing_document = billing_document`. The resolver finds the shortest path from the base table to every other needed table, merges those paths (removing duplicates), and assembles the JOIN chain.

The `max_join_depth` of 3 is a safety constraint. If a query would require 4+ JOINs in PostgreSQL, it's probably a multi-hop traversal that should go to Neo4j instead. The router would catch this case if properly classified, but the join resolver acts as a second safety net — if you somehow get a 4-join SQL query, something went wrong in routing.

Part 4: The Cypher Builder

For `FlowTraceIntent` and `BrokenFlowIntent`, the Cypher builder translates the

For `flow_trace_intent` and `broken_flow_intent`, the Cypher builder translates the intent into graph traversal queries against Neo4j.

Log in to see interactive visuals

The Cypher builder has a parallel safety model to the SQL builder. Node labels (SalesOrder, Payment, Delivery) come from the registry's entity catalog, relationship types (HAS_DELIVERY, BILLED_BY) come from the registry's join paths, and all values are parameterized. The `max_depth` cap (from the intent, capped at system max of 6) prevents runaway traversals that could scan the entire graph.

The key difference from the SQL builder: the Cypher builder also returns **path structure** (which nodes and edges were traversed), not just flat rows. This is what makes Neo4j valuable for your use case — the prose generator can say "Order 12345 was delivered via delivery 80001234, billed as invoice 90001234, and paid through clearing document 14001234" because it has the full chain.

Part 5: Edge Cases and Failure Modes

This is where production systems live or die. Let me map out every failure point I can see in the query builder:

Log in to see interactive visuals

The **ambiguous join path** one deserves extra attention because it's specific to your SAP data. Look at your ER diagram: `billing_document_headers` connects to `business_partners` through `sold_to_party`, but you could also reach `business_partners` through `journal_entry_items_accounts_receivable.customer`. These two paths give different business meanings — one is "who did we bill?" and the other is "who owes us money in accounting?" The registry must mark one as the **preferred path** per context, and the intent's `entity_type` helps disambiguate. If the base is billing data, use the `sold_to_party` path. If the base is accounting data, use the journal entry path.

Part 6: The Design Pattern

The query builder itself follows the **Builder pattern** — not the GoF creational builder, but a fluent clause-assembly builder. Each method adds one clause (`.select()`, `.from_table()`, `.join()`, `.where()`, `.group_by()`, `.order_by()`, `.limit()`), and `.build()` produces the final `(query_string, params_tuple)`. The SQL builder and Cypher builder are separate concrete builders behind a common `QueryBuilder` interface — the store router picks the right one.

This is exactly the kind of thing your coding agent will need explicit instructions for:

This is exactly the kind of thing your coding agent will need explicit instructions for: the builder must be **stateless between calls** (each intent gets a fresh builder instance), and the `.build()` method must do a final validation pass — checking that SELECT fields match GROUP BY when aggregating, that ORDER BY references a valid output column, and that LIMIT is always present.

Layer 2 Summary:

Component	Responsibility	Pattern
Store Router	Intent → PostgreSQL or Neo4j	Strategy
SQL Builder	Clause-by-clause assembly with parameterization	Builder
Cypher Builder	Graph traversal pattern assembly	Builder
Join Resolver	BFS pathfinding on schema join graph	Graph algorithm
Safety layer	Type checks, depth limits, timeouts	Validation chain

One thing I want you to note for your coding agent instructions: the join resolver's BFS pathfinding is the single most testable component in the entire system. You can unit test it exhaustively with your 18 entities and their known join paths — no LLM, no database, just "given tables A, B, C needed, what JOINs should be produced?" That's the kind of test that catches bugs before they ever hit production.

Q: Ready to move on to Layer 3, or want to go deeper on anything in the query builder?
A: All clear — move to Layer 3: PostgreSQL + Neo4j data model

Layer 3 is where your abstract domain model meets physical storage. We need to answer three questions: how do the 18 SAP entities map to PostgreSQL tables, how does the same data project into Neo4j as a graph, and how do you keep them in sync?

Part 1: PostgreSQL Schema Design

Your ER diagram already gives us the table structure. The key design decisions are around FK enforcement, indexing strategy, and how to handle SAP's peculiarities.

Log in to see interactive visuals

A few important PostgreSQL design decisions to call out.

Composite primary keys — SAP data naturally uses composite keys (e.g., sales_order + sales_order_item together identify a line item). We should honor that in PostgreSQL rather than adding surrogate IDs. This preserves the SAP document numbering that your users already know and query by. The LLM will receive document numbers from users like "order 12345" — those map directly to the PK.

Indexing is driven by your intent types — EntityList queries filter by customer + date range most often, so that gets a composite index. Aggregation queries group by material and sum net_amount, so that combination gets indexed. Soft lookup columns (reference_sd_document) get their own indexes because the join resolver will use them heavily. We're not guessing — we're indexing exactly what the query builder will produce.

SAP string fields — SAP pads document numbers with leading zeros (e.g., "0000012345"). Your alias resolution layer should handle both "12345" and "0000012345" mapping to the same document. This is a normalization step that happens after alias resolution but before query building.

Part 2: Neo4j Graph Model

This is where the same data takes a completely different shape. In PostgreSQL you have tables with rows. In Neo4j you have nodes with relationships. The mapping isn't one-to-one — some PostgreSQL tables become nodes, others become relationships, and some become properties on existing nodes.

Log in to see interactive visuals

This is a critical modeling decision that I want to make sure you understand fully. **In PostgreSQL, item tables are first-class tables. In Neo4j, they collapse into relationship properties.** This isn't just aesthetic — it directly affects your Cypher query depth. If you made `SalesOrderItem` a node, then tracing "Customer → Order → Delivery" would be Customer → SalesOrder → SalesOrderItem → DeliveryItem → Delivery — that's 4 hops instead of 2. Your `max_depth` would need to be much higher, and every traversal query gets slower.

The exception: if you ever need to query line items independently (e.g., "find all line items for material XYZ across all orders"), that's a PostgreSQL query, not a Neo4j traversal. The store router handles this — item-level analytics go to PostgreSQL, flow traversals go to Neo4j.

Node properties — each node carries only the fields needed for graph-level filtering

Node properties — each node carries only the fields needed for graph-level filtering and display. A `SalesOrder` node has `id`, `creation_date`, `total_net_amount`, `sold_to_party`, `status`. It does not carry every column from `sales_order_headers` — fields like `created_by_user` or `transaction_currency` stay in PostgreSQL only. The graph is lean; the relational store is complete.

Part 3: The Sync Strategy

This is the operational heart of your dual-store architecture. PostgreSQL is truth. Neo4j is a projection. When PostgreSQL changes, Neo4j must follow.

Log in to see interactive visuals

Let me unpack the critical design decisions here.

Why `updated_at` polling instead of Change Data Capture (CDC)? For hundreds of customers with moderate data volume, CDC (like PG logical replication or Debezium) is overengineered. A simple `updated_at` timestamp on every table, with a sync worker that runs every 10-30 seconds, gives you near-real-time sync with minimal infrastructure. You add one column and one index per table. If you later scale to hundreds of thousands of customers, you can swap the change detector for CDC without touching the transformer or writer — that's the beauty of the pipeline architecture.

The Transformer is where the modeling happens. When the change detector says "row updated in `sales_order_headers`," the transformer knows to MERGE a `(:SalesOrder)` node in Neo4j. When `sales_order_items` changes, it doesn't create a node — it MERGEs a `[:CONTAINS_ITEM]` relationship between the `SalesOrder` and `Product` nodes, carrying `quantity` and `amount` as relationship properties. This transformation logic is essentially a mapping configuration, declared alongside the schema registry.

Handling SAP's orphaned references — this is unique to your data. When `billing_document_items.reference_sd_document` points to a sales order that doesn't exist in your system (maybe it was archived, or it's from a different system), the transformer should skip that relationship creation rather than creating a dangling edge. It logs the orphan for investigation but doesn't break the graph. This is why we keep soft lookups in PostgreSQL — the graph only contains relationships that actually resolved.

Part 4: How the two stores serve different query patterns

Let me make the division of labor explicit:

Log in to see interactive visuals

Layer 3 Summary:

Component	Decision	Rationale
PG primary keys	Composite, SAP-native	Users query by document numbers
FK strategy	Hard for header↔item, soft for cross-doc	SAP data quality allows orphans
Neo4j nodes	Header tables + master data (8 types)	Keep graph shallow
Neo4j relationships	Item tables collapse to relationship properties	2 hops instead of 4
Sync mechanism	`updated_at` polling, 10-30s interval	Right-sized for hundreds of customers
Sync failure	Queue + retry, staleness indicator in response	Graceful degradation
Schema version	Tagged on both stores, mismatch blocks + resyncs	Prevents stale graph queries

One important architectural note for your coding agent: the **transformer configuration** (which PG tables map to which Neo4j node types and relationships) should live alongside the schema registry, not in a separate file. When you add a new entity or change a relationship in the registry, the sync transformer should automatically know about it. One source of truth, not two that drift apart.

Q: How's Layer 3 sitting with you? Ready for the LLM adapter design?
A: All clear — move to Layer 4: LLM adapter + fallback chain

Layer 4 is where you interface with the most unpredictable component in your system — the LLM. The entire design philosophy here is: **assume every LLM call can fail, and make the system resilient to that.**

Your system makes two fundamentally different types of LLM calls, and each has different reliability requirements. Intent extraction needs structured output — it must return valid JSON that Pydantic can parse. Prose generation is more forgiving — it

return valid JSON that Pydantic can parse. Prose generation is more forgiving — it just needs coherent text. This distinction drives the entire adapter design.

Part 1: The Adapter Interface

Log in to see interactive visuals

The interface has three methods, and each adapter implements them differently. `generate_structured` is the critical one — it takes a prompt and a Pydantic schema, and returns a validated instance. Gemini has native JSON mode with schema enforcement. Groq supports JSON mode but with weaker schema adherence. OpenRouter varies by underlying model. Each adapter handles these differences internally — the caller never knows which provider is running.

`health_check` returns a `ProviderHealth` object containing: is the provider reachable, what's the current rate limit remaining, what's the average latency over the last N calls, and when was the last successful call. This feeds directly into the fallback chain.

Part 2: The Fallback Chain

This is where your "auto-rotation on credit exhaustion" requirement meets reality. The fallback chain isn't just about rotation — it's a decision engine that picks the best available provider for each call type.

Log in to see interactive visuals

Let me walk through a concrete failure scenario so this is tangible.

User asks "Top 5 products by revenue." The system picks Gemini (healthy, highest priority). Gemini returns a 429 (rate limited). The health registry increments Gemini's consecutive fails to 1, and immediately re-selects — Groq is degraded but alive. Groq attempts the call, returns valid JSON, the request succeeds. Gemini's consecutive fails stay at 1 (not reset until it succeeds again). If Gemini had failed 3 times in a row, it would be marked dead and put on cooldown. Meanwhile, every successful Groq call keeps Groq's health metrics fresh.

The **JSON reliability score** is a subtlety worth calling out. You track, per provider, what percentage of `generate_structured` calls return parseable, schema-valid JSON. Gemini might score 95%, Groq 80%, a cheap OpenRouter model 60%. For intent extraction (which must produce valid JSON), you prefer providers with higher reliability scores. For prose generation (which just needs text), reliability doesn't matter — you optimize for speed and cost.

Part 3: Structured Output Handling

This is the most failure-prone part of the entire LLM layer. Even good models sometimes return malformed JSON, extra text around the JSON, or valid JSON that doesn't match your Pydantic schema. The adapter needs a multi-stage parser.

Log in to see interactive visuals

There's a subtle but important retry distinction here. If JSON extraction fails (stage 1-2), the model likely wrapped its response in markdown or added commentary. You retry with the same provider but add a stronger instruction: "Return ONLY a JSON object, no other text." If Pydantic validation fails (stage 3-4), the model returned valid JSON but with wrong structure — maybe it used `order_id` instead of `order_number`. You retry with the error message injected: "Your response was invalid: field 'order_id' is not a known alias. Available aliases for order identifiers are: order_number, sales_order_number."

This error-feedback retry is powerful because the LLM learns from its mistake within the same request. Most models self-correct on the second attempt when told exactly what went wrong.

Part 4: Prompt Architecture

The prompts for intent extraction and prose generation are fundamentally different, and both need careful design.

Log in to see interactive visuals

Two critical prompt design principles to highlight.

The schema context is auto-generated, not hand-maintained. When you add a new entity or alias to the registry, the prompt template pulls from the same registry and rebuilds the context section. This eliminates the drift problem where your registry supports a field but the LLM prompt doesn't mention it. One source of truth, enforced programmatically.

The prose generation prompt has grounding rules that prevent hallucination. The LLM sees the raw data and is explicitly told: "only describe what the data shows, do not speculate or infer." If the query returned zero rows, it must say "No matching records found" not "There might not be any orders because..." This is your defense against the LLM inventing business insights.

Part 5: Observability — what to log at the LLM layer

Every LLM call generates a `LLMCallRecord` that the supervision layer observes. This

Every LLM call generates a `LLMCallRecord` that the supervision layer observes. This is critical for debugging, cost tracking, and quality improvement.

Log in to see interactive visuals

This observability data creates a feedback loop. If you notice Groq's `json_validity_rate` dropping below 70%, you can proactively lower its priority for structured calls before it starts causing user-facing failures. If `auto_corrections_count` spikes for a particular alias pattern (say users keep triggering "invoice_amount" → corrected to "billing_amount"), that's a signal to add "invoice_amount" as an official alias in the registry. The system gets smarter over time without requiring code changes.

Layer 4 Summary:

Component	Decision	Pattern
Interface	`LLMClient` with `generate_structured` + `generate_text` + `health_check`	Adapter
Providers	Gemini (primary), Groq (secondary), OpenRouter (tertiary)	Configurable priority
Fallback	Health-based selection → retry on failure → next provider	Chain of Responsibility
Health tracking	In-memory per-instance, 3 consecutive fails → dead → cooldown	Circuit Breaker
Structured parsing	4-stage pipeline: extract → parse → validate → resolve	Pipeline
Retry strategy	Stage 1-2 fail: retry with stricter instruction. Stage 3-4: retry with error feedback	Error-guided retry
Prompt management	Auto-generated from registry, few-shot examples versioned	Template + registry
Observability	Per-call records + aggregated metrics feeding back into selection	Observer

One architectural insight for your coding agent: the `LLMClient` interface should be **async throughout** since you're using FastAPI. Every `generate_structured` and `generate_text` call is an HTTP request to an external API — these must be `async def` methods using `httpx.AsyncClient` or the provider's async SDK. This way

`def` methods using `httpx.AsyncClient` or the provider's async SDK. This way, FastAPI's event loop isn't blocked while waiting for LLM responses, and you can later support concurrent requests trivially.

Q: Ready for Layer 5, or need to dig into any part of the LLM layer?
A: All clear — move to Layer 5: Intent router + handlers

Layer 5 is the orchestration core — the component that receives a validated intent and coordinates everything: picking the right handler, calling the query builder, executing against the right store, and passing results to prose generation. It's the glue between every other layer.

The design philosophy here: **the router is dumb, the handlers are smart.** The router's only job is dispatch — mapping an intent type to a handler. Each handler owns the full lifecycle for its intent type: building the query, executing it, shaping the results, and preparing context for prose generation.

Part 1: The Router Architecture

Log in to see interactive visuals

I want to call out the disagreement with your BAES doc directly. You mentioned Chain of Responsibility for intent routing. CoR makes sense when handlers need to *inspect* a request and decide if they can handle it — like guardrails where each guard says "pass" or "reject." But intent routing has a discriminator field (`intent_type`) that's already been validated by Pydantic. There's zero ambiguity. A simple `{intent_type: handler_class}` dictionary is cleaner, faster, and easier to debug. Save CoR for where it actually earns its keep — the guardrail chain in Layer 6.

Part 2: The Handler Interface and Lifecycle

Every handler follows the same contract. This is where the Strategy pattern genuinely applies — each handler is a different strategy for fulfilling a different intent type, but they all share the same interface.

Log in to see interactive visuals

The `shape_result` step is the key differentiator between handlers. Let me walk through what each handler does differently here, because this is where domain logic lives.

Part 3: Handler-Specific Behavior

Each handler has different query-building logic and — critically — different result-shaping logic. The prose LLM needs different context depending on whether it's describing a single entity, a ranking, or a graph path.

Log in to see interactive visuals

The `shape_result` method is doing something important that's easy to underestimate. The prose LLM doesn't need all 500 rows from an EntityList query — it needs a concise summary like "Found 47 orders for customer ACME. The most recent was Order 12345 on March 15, totaling $14,200. The oldest undelivered order is Order 10001 from January 3." That narrative is built from a shaped context, not raw data. The raw data goes to the API response for the frontend to display in tables or graphs.

Part 4: The Compound Handler — the orchestrator

This deserves special attention because it's the most complex handler. It doesn't call the query builder itself — it delegates to other handlers and manages the data flow between them.

Log in to see interactive visuals

The reference resolver is the only piece of the CompoundHandler that has real logic. It parses `$step_1.results[0].product` into a path — step ID, result index, field name — and extracts the value from stored results. The path syntax is deliberately simple and rigid: `$step_N.results[index].field_name`. No nested access, no expressions, no wildcards. If the reference can't resolve (step failed, index out of range, field missing), the compound execution stops at that step and returns partial results.

Part 5: The QueryService Facade

This is the top-level orchestrator that ties the entire request lifecycle together. From the outside, it's a single `async def answer(question: str)` method. Inside, it coordinates every layer.

Log in to see interactive visuals

The Facade pattern here isn't just organizational — it's strategic for your coding agent workflow. Your `instructions.md` can say "the API endpoint calls

`QueryService.answer(question)` and returns the result." The agent doesn't need to

understand the internal wiring to build the API layer. And when you want to add a new intent type later, you only touch three things: add a Pydantic model to the DSL, write a new handler class, and register it in the router dict. No other layer changes.

Layer 5 Summary:

Component	Pattern	Responsibility
IntentRouter	Registry (dict lookup)	Map `intent_type` → handler class
BaseHandler	Strategy (interface)	`handle()`, `build_query()`, `shape_result()`
EntityLookup/ListHandler	Strategy (concrete)	PG queries for single/multi entity
AggregationHandler	Strategy (concrete)	PG GROUP BY queries
FlowTraceHandler	Strategy (concrete)	Neo4j path traversal
BrokenFlowHandler	Strategy (concrete)	PG LEFT JOIN or Neo4j OPTIONAL MATCH
CompoundHandler	Composite + Mediator	Orchestrates sub-handlers with $ref resolution
QueryService	Facade	Single entry point for entire pipeline

One design insight worth highlighting: every handler is stateless. They receive an intent, produce a result, and hold nothing between calls. This means handlers can be instantiated once at startup and reused across all requests — no per-request allocation overhead. The `RequestContext` carries per-request state (audit trail, timing), not the handlers.

Q: Ready for the final layer — supervision and guardrails?
A: All clear — move to Layer 6: Supervision + guardrails

Layer 6 is your safety net, your debugger, and your quality monitor — all in one. It runs alongside every other layer without interfering with business logic. The design philosophy: **the system should be fully explainable after the fact.** For any request,

you should be able to reconstruct exactly what happened, why, and where it went

you should be able to reconstruct exactly what happened, why, and where it went wrong if it did.

Three sub-systems here: the Event Observer stream (what happened), the Guardrail Chain (what's allowed), and the Audit Trail (the permanent record).

Part 1: The Event Observer Stream

Every significant action in the pipeline emits an event. Observers subscribe to these events and do their thing — logging, metrics, real-time streaming to the UI — without the core pipeline knowing or caring about them.

Log in to see interactive visuals

The critical design rule for observers: **they are fire-and-forget and never block.** If the MetricsObserver throws an error, the user's query still completes. Observers wrap their logic in try/except internally and swallow errors to a fallback logger. The pipeline's job is answering questions — observability is important but not at the cost of availability.

The `AuditObserver` deserves special attention. It doesn't write to the database on every event — it accumulates all events for a request in memory, and on the `completed` or `error` terminal event, it assembles a single `AuditRecord` and persists it in one write. This avoids N database writes per request and gives you a complete, atomic record.

Part 2: The Guardrail Chain

This is where Chain of Responsibility genuinely earns its place — unlike intent routing where I pushed back on it. Each guardrail inspects the request context and either approves (pass to next) or rejects (terminate with friendly message). The chain runs after intent parsing but before query execution.

Log in to see interactive visuals

The chain ordering is intentional and cost-optimized. Domain scope check is almost free — it just checks `intent_type`. Field validation requires a registry lookup but no I/O. Operator-type compatibility is pure logic. Complexity limiting is arithmetic. Rate limiting is last because it's the only guard that needs external state (a counter in Redis or memory). If a request fails at Guard 1, you never pay the cost of Guard 5.

Rejection messages are user-facing, not developer-facing. When Guard 2 rejects because the LLM hallucinated a field called `invoice_status`, the user shouldn't see "Field 'invoice_status' not found in registry." They should see: "I couldn't find a field called 'invoice status' in the billing data. Did you mean 'billing document type' or 'goods movement status'?" The guardrail produces both a machine-readable rejection

goods movement status." The guardrail produces both a machine-readable rejection code (for logs) and a human-friendly message (for the response). If fuzzy matching found close aliases, those are included as suggestions.

Part 3: The Audit Trail

This is your permanent record. Every request gets an `AuditRecord` that captures the full lifecycle — what the user asked, what the LLM produced, what query ran, what data came back, and what the final answer was. This is essential for debugging, quality evaluation, and explaining system behavior.

Log in to see interactive visuals

Two important storage decisions for the audit trail. First, `intent_raw_json` and `intent_validated` are stored as separate columns — not just the validated version. This lets you see exactly what the LLM produced versus what the system corrected. If you only stored the corrected version, you'd lose visibility into LLM quality over time. Second, `query_generated` stores the actual SQL/Cypher string with parameter placeholders, and `query_params` stores the values separately. This means you can replay any historical query against the database if you need to debug a data issue.

The audit table lives in PostgreSQL alongside your O2C data but in a separate schema (e.g., `audit.request_logs`). It has its own retention policy — you might keep 30 days of detailed records and archive older ones — so it doesn't bloat your primary data tables.

Part 4: Putting it all together — the supervision layer's integration points

Log in to see interactive visuals

The feedback loops are what make supervision a living system rather than a passive logger. It's not just recording what happened — it's feeding signals back into the layers that need to adapt. The provider health registry in Layer 4 is constantly updated by metrics from Layer 6. The alias registry in Layer 1 receives suggestions from correction patterns observed in Layer 6. This is the system getting smarter over time without code changes.

Layer 6 Summary:

Component	Pattern	Responsibility
EventBus	Observer / Pub-Sub	In-process event dispatch to subscribers

Component	Pattern	Responsibility
RequestContext	Subject (emitter)	Carries per-request state, emits lifecycle events
LoggingObserver	Observer (concrete)	Structured JSON logs per event
MetricsObserver	Observer (concrete)	Counters, histograms, gauges for dashboards
AuditObserver	Observer (concrete)	Assembles complete AuditRecord on completion
Guardrail chain	Chain of Responsibility	Ordered validation: scope → fields → types → complexity → rate
AuditRecord	Data transfer object	Full request trace persisted to PG

That's all six layers designed. Let me give you the complete architecture at a glance — every pattern, every decision, every layer connected.

Log in to see interactive visuals

That's the complete LLD — all six layers designed, every pattern justified, every edge case identified.

What you now have and what comes next:

You have a fully reasoned architecture covering DSL contracts, query construction, dual-store data model, LLM resilience, handler orchestration, and supervision. Every design decision is traceable to a concrete requirement from your BAES document and your SAP O2C domain.

For your coding agent, you'll need to translate this into three artifacts: an `instructions.md` covering the project structure and layer-by-layer implementation order, a `brandguidelines.md` (coding practices — async everywhere, Pydantic for all boundaries, structured logging, etc.), and crucially the **alias dictionary** mapping your SAP field names to semantic names — that's pure domain knowledge only you can provide.

My recommended implementation order for the coding agent: Schema Registry first (it's the foundation everything references), then PostgreSQL schema + migrations, then the DSL Pydantic models, then handlers + query builder, then LLM adapters, then the FastAPI facade, then Neo4j sync, and supervision last (it wires into everything but blocks nothing).

Q: What do you want to do next?
A: Generate the instructions.md for my coding agent

Let me first understand the skill for creating this document, then I'll build a comprehensive `instructions.md` that captures every architectural decision we've made.

Reading the docx skill for document creation best practices >

Now let me build the comprehensive `instructions.md`. This is going to be substantial — it needs to capture every decision across all 6 layers so your coding agent can implement without ambiguity.

Ran 2 commands, created a file, read a file >

That's ~1,000 lines covering every architectural decision we made across all 6 layers. Here's what the document contains and what you still need to provide yourself.

What's in the instructions.md:

The complete project structure (every file and its responsibility), implementation order in 6 phases, all 18 entity definitions with field types, the full alias registry structure with examples, join path graph with all edges, every Pydantic intent model with field types, the SQL and Cypher builder specifications, PostgreSQL schema design with indexing strategy, Neo4j graph model with node-to-relationship mapping, sync pipeline design, LLM adapter interface with fallback chain algorithm, prompt templates for both intent extraction and prose generation, guardrail chain with all 5 guards, event bus and observer specifications, API endpoint contracts, and 9 documented edge cases.

What you need to add before handing to your coding agent:

1. **Complete alias dictionary** — I provided ~20 example mappings, but you need to cover all fields across all 18 entities. Only you know which SAP field names your users actually say in natural language.
2. **Your `brandguidelines.md`** — your coding practices. I included an implicit section on async, error handling, and type safety, but your agent needs your personal conventions for naming, commit style, docstrings, etc.
3. **LLM API keys** — for Gemini, Groq, and OpenRouter. The `.env.example` structure is defined.
4. **Few-shot examples** — I described the structure (1-2 per intent type, ~10 total), but the actual example pairs ("user question" → "expected JSON output") should be written by you since you know what your users actually ask.

be written by you since you know what your users actually ask.

```
# LLM Provider Configuration (gemini | groq | openrouter)
LLM_PROVIDER=openrouter

# Google Gemini Configuration
GEMINI_API_KEY=AIzaSyBg2iu7CPT4GZTDumAJ5ICJ0WChhlJ8n5Q
GEMINI_MODEL=gemini-2.0-flash
```

Show more

Important security note first: You've shared actual API keys in the chat. I'd strongly recommend rotating those keys immediately — Gemini, Groq, and OpenRouter all allow key regeneration in their dashboards. Treat any key shared in a conversation as compromised.

Now let me integrate your brand guidelines and env configuration into the project artifacts.

Creating .env.example with placeholder values (never store real keys) >

Now let me create a proper `brandguidelines.md` that merges your brand identity with the engineering coding practices we discussed:

Created a file, ran a command >

Now let me update the `instructions.md` to reference GraphIQ and integrate the provider configuration:

Edited 5 files, ran a command, read a file >

Here's your complete project handoff — three files ready for your coding agent:

`instructions.md` (~1,000 lines) — the complete implementation blueprint covering all 6 layers, project structure, every Pydantic model, query builder specs, sync pipeline, LLM adapter design, guardrail chain, and 9 documented edge cases. Updated with GraphIQ branding and your specific LLM model choices.

`brandguidelines.md` — your product identity merged with engineering standards: naming conventions, async discipline, error handling philosophy, dependency

injection rules, testing standards, logging format, git practices, and the "what NOT to

injection rules, testing standards, logging format, git practices, and the "what NOT to do" list. This is what makes your codebase look like an expert built it.

`.env.example` — complete configuration template with all environment variables, including fuzzy matching thresholds, LLM health tracking parameters, and system limits. Placeholder values only — no real keys.

Three things to do before handing these to your agent:

1. **Rotate your API keys immediately** — the ones you shared in chat should be considered compromised. Regenerate them in Gemini Console, Groq Console, and OpenRouter dashboard.
2. **Expand the alias dictionary** in `instructions.md` — I provided ~20 example mappings. You need to cover all fields across all 18 entities with the natural language terms your users actually use.
3. **Write the few-shot examples** — 1-2 per intent type (~10 total). Use real questions from your domain: "Show me order 12345", "Top products by billing", "Trace order to payment", etc. paired with the exact JSON the LLM should produce.

Files hidden in shared chats

now give me prompt

these are the main features that i want because what we discussed is very high level and it is not needed at that extent

Good call. The full 6-layer architecture is the target — but your coding agent needs a focused, buildable scope for v1. Let me create a prompt that keeps our architectural decisions as the backbone but scopes the execution to what actually ships.

Created a file, ran a command, read a file >

Here's your complete project handoff — four files working together:

`graphiq_agent_prompt.md` (335 lines) — this is the execution prompt your coding agent reads first. Here's how it differs from your original Anti-Gravity template:

It's not generic anymore. Every instruction is GraphIQ-specific — the exact dataset

It's not generic anymore. Every instruction is GraphiQ-specific — the exact dataset entities are listed, the O2C flow is defined, the 20-step build order maps directly to our 6-layer architecture, and the test cases at the end are real queries your system must handle. The operating rules from your template are preserved but sharpened: "the LLM does exactly two things: parse intent and write prose" is now an explicit constraint, not implied.

The build order is structured into 6 phases (Foundation → Core Engine → LLM Integration → Graph Layer → API + Frontend → Supervision) with each step numbered and specific about what to build. Your agent knows exactly when to create the schema registry vs when to wire the frontend.

How the four files work together:

The agent reads `graphiq_agent_prompt.md` first — it tells them *what to build and in what order*. When they need architectural detail (Pydantic model definitions, join path specs, query builder internals), they reference `instructions.md`. When they need coding style decisions (naming, error handling, logging, git), they reference `brandguidelines.md`. The `.env.example` gives them the configuration template.

What you still need to provide alongside these files:

1. The actual dataset files (CSVs/JSONs) for the 19 SAP entities
2. Your rotated API keys in a real `.env` file
3. The few-shot examples (1-2 per intent type — real question → expected JSON pairs)
4. The complete alias dictionary (expand the ~20 examples in instructions.md to cover all fields)

Files hidden in shared chats

gow this implementation plan

This is solid — your coding agent clearly absorbed the architecture. It understood the schema registry as foundation, the handler pattern, the fallback chain, the store router, and the build order. That means the instruction files we wrote are doing their job.

But I see several issues that will cause problems during implementation. Let me give

But I see several issues that will cause problems during implementation. Let me give you a precise review.

The agent did a good job absorbing the architecture. It got the core flow right — schema registry first, handlers following the Strategy pattern, parameterized queries, fallback chain. But there are specific issues that will cause implementation failures if not corrected. Let me give you a precise scorecard.

Log in to see interactive visuals

Now let me go through each issue with the exact correction you should feed back to your agent.

Issue 1: Project root name. The plan says `o2c_query_system/`. We renamed it to `graphiq/`. This seems small but affects every import path. Tell the agent: "Project root is `graphiq/`, not `o2c_query_system/`. Update all references."

Issue 2: Missing few-shot examples. The plan mentions `intent_extraction.py` but doesn't mention that the prompt needs 1-2 few-shot examples per intent type baked in. Without these, the LLM's structured output reliability will drop significantly on the first attempt, triggering more retries and fallbacks. Tell the agent: "The intent extraction prompt MUST include few-shot examples — at least one example per intent type showing a user question → expected JSON output pair. These are hardcoded in the prompt builder, not dynamic."

Issue 3: Prompt auto-generation from registry. The plan says `build_intent_prompt(registry, question)` which is the right signature, but doesn't emphasize that the schema context section (available entities, fields, aliases, operators) is AUTO-GENERATED from the registry at call time. This is critical — if the registry changes, the prompt must change automatically. Tell the agent: "The schema context section of the intent prompt must be programmatically generated from `SchemaRegistry` every time, not hand-written as a string. When a new alias or entity is added to the registry, the prompt updates without code changes."

Issue 4: SAP document number normalization. The ingestion script (`scripts/ingest_data.py`) is mentioned but doesn't specify SAP zero-padding. Document numbers like `12345` must become `0000012345`. This affects joins, lookups, and user queries. Tell the agent: "The ingestion script must normalize all SAP document number fields to 10-digit zero-padded strings. The alias resolver must also pad user-provided document numbers before query execution — if a user says 'order 12345', it should resolve to '0000012345'."

Issue 5: `sync_lag_seconds` missing from API response. Our architecture specifies that every query response includes the Neo4j sync lag so the prose generator can note data freshness. The plan's API response schema doesn't include this. Tell the agent:

"The `/query` response metadata must include `sync_lag_seconds` (difference

The `/query` response metadata must include `sync_lag_seconds` (difference between now and last successful Neo4j sync). The prose generation prompt should mention this when lag > 60 seconds."

Issue 6: Cross-currency aggregation edge case. When aggregating monetary fields (net_amount, total_net_amount), the system must not blindly sum across different currencies. The AggregationHandler should detect when group_by doesn't include a currency field while the measure is monetary, and either auto-add currency to group_by or warn in the prose context. Tell the agent: "AggregationHandler must check: if the measure field is a monetary amount AND group_by does not include a currency field, add a warning to prose_context: 'Note: results may span multiple currencies.' Better yet, auto-add `transaction_currency` to group_by when aggregating monetary fields."

Issue 7: Structured parser retry detail. The plan mentions the 4-stage pipeline but doesn't detail the error-feedback retry — when Pydantic validation fails (stage 3-4), the system retries with the specific error injected into the prompt so the LLM can self-correct. Tell the agent: "When structured parsing fails at stage 3 or 4, the retry must inject the error into the prompt: 'Your previous response was invalid: {specific_error}. Available aliases for this context: {relevant_aliases}.' This error-guided retry is what makes weaker models viable · [illegible] ond attempt."

Start your own conversation

Issue 8: Frontend graph ↔ chat sync. The plan lists GraphView and ChatPanel as separate components but doesn't specify how a query result highlights nodes in the